



Actividad práctica en parejas

Objetivo: usar `abstract` para prohibir lo que no existe y obligar a completar lo que falta, y una interfaz para modelar una capacidad que atraviesa la jerarquía.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya está clonado, alcanza con actualizarlo desde la carpeta del repositorio:

```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/P00-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es `06-abstractas-e-interfaces/actividad`.

Es la jerarquía de la clase pasada ya terminada, y compila y corre tal como viene.

- `Producto.java` es la superclase. Se modifica en la parte 2.
- `Libro.java`, `Electronico.java` y `Alimento.java` son las tres subclases. Se modifican en las partes 2 y 3.
- `Suscripcion.java` no hereda de nadie. Se modifica en la parte 3.
- `Descontable.java` no existe todavía. Se escribe en la parte 3.
- `Tienda.java` es el programa que muestra el catálogo.

Parte 1: ver los dos problemas

Ejecutar `Tienda` tal como viene y mirar la salida completa. Responder entre los dos, por escrito y en pocas palabras.

1. El último ítem del catálogo es un “Artículo de bazar” que no es un libro, ni un electrónico, ni un alimento. ¿De qué clase es? ¿Por qué el compilador dejó crearlo?
2. Supongamos que libro está exento de IVA y los alimentos de la canasta básica tributan un porcentaje reducido del 10,5%. El catálogo les cobra 21% a los tres. ¿En qué archivo está el cálculo y cuántas veces está escrito?
3. Cuando se escribió `Alimento`, ¿el compilador podía avisar que faltaba definir su IVA? ¿Por qué?

Parte 2: arreglar con `abstract`

Los pasos están marcados como `TODO` dentro de los archivos. En orden.

- `TODO 1 (Producto)`: declarar la clase `abstract`.
- `TODO 2 (Producto)`: sacar la constante `IVA` y dejar `getPrecioFinal()` declarado, sin cuerpo y marcado como `abstract`. Nada más cambia en la clase.
- `TODO 3 (Libro)`: declarar su `IVA`, que es cero, e implementar `getPrecioFinal()`.



- TODO 4 (Electronico): lo mismo con el 21%.
- TODO 5 (Alimento): lo mismo con el 10,5%.
- TODO 6 (Tienda): borrar la línea que creaba el artículo de bazar. Ahora no compila, y ese es el punto.

Antes de ejecutar, responder: ¿qué error marca el compilador en cada subclase apenas se hace el TODO 2, y por qué es exactamente el aviso que faltaba en la parte 1?

Y una para mirar el código: `mostrarDetalle()` quedó escrito en `Producto` y llama a `getPrecioFinal()`, lo `Producto` no le da un cuerpo. ¿Por qué funciona?

Parte 3: un “sabe hacer” más allá de la herencia

La tienda hace una liquidación. Se liquidan los libros (edición vieja) y los alimentos (vencimiento cercano), pero no los electrónicos, cuyo precio lo fija el fabricante. Y también se liquida la suscripción al club de lectura, que no es un producto del catálogo.

- TODO 7: crear `Descontable.java` con la constante `DESCUENTO_MAXIMO` en 50 y dos métodos, `aplicarDescuento(int porcentaje)`, que devuelve boolean, y `getPrecioConDescuento()`.
- TODO 8 (Libro) y TODO 9 (Alimento): agregar `implements Descontable` junto al `extends` que ya tienen, el atributo del porcentaje y los dos métodos. Rechazar el descuento fuera de rango devolviendo `false`, sin cambiar el estado. En `Alimento`, borrar la constante `DESCUENTO` y la línea del precio de oferta.
- TODO 10 (Suscripcion): implementar el mismo contrato. Esta clase no hereda de `Producto`, así que calcula sobre su propio precio mensual.
- TODO 11 (Tienda): escribir `liquidar(Descontable[] items, int porcentaje)`, que recorre el arreglo, aplica el descuento e imprime el precio que queda. Después descomentar el bloque final de `main` y ejecutar.

`Electronico` no se toca en toda esta parte. Para cerrar: agregar la notebook al arreglo `enLiquidacion` y ver qué dice el compilador. Esa línea de error es el argumento entero de la clase de hoy.